

22A – M2 Catalogue & Batch: Implementation Guide

Everything the M2 developer needs: what it is, what you inherit, the schema as runnable DDL, seed data, package layout, the API slice, every command as a handler specification, the duplicate-detection and assortment algorithms, the snapshot contributor, screens, tests and the tickets in build order. Self-contained; restated material is marked.

Field	Value
Document	22A – implementation guide for 22 M2 Catalogue & Batch v0.1
Version	v0.1 – for the M2 developer; starts against M1's frozen contract
Date	8 September 2026
Prerequisite	Sprint 0 (17A, 19A) and M1's api and query packages (21A ticket M1-13). M3's price-list query and M5's lot query are stubbed until their modules land (Section 11).
Estimated effort	About 21 developer-days for one full-stack developer (Section 10)
Restated content	Marked <i>restated from doc X §Y</i> .

Read this first. Two things in M2 are harder than they look and are where the time goes: the steward's import-and-deduplicate tooling (Section 6.2), and the assortment maintainer that decides what each of 10,000 shops' tills must know (Section 7.2). Build the plain aggregates first (tickets M2-01 to M2-05), then those two, then screens. Do not start with screens.

1. What you are building

Restated from doc 22 §1. M2 defines what can be counted and sold: SKU identity in one global namespace, units and conversions, barcodes, tags, tax categories, images, and the batch as physical identity carrying expiry and printed MRP. It owns identity, not holding; stock and cost are M5.

You build	You do not build
Schema catalogue: uom, sku, sku_uom_conversion, sku_barcode, sku_alias, sku_image, tag, sku_tag, tax_category, tax_rate, batch, supplier, location_assortment, duplicate_suspect	Stock lots, movements, costs (M5); price lists, control prices (M3)
17 commands, 11 queries, 9 event families (Section 5)	Repack recipes and execution (M5); the till's scan layer (M6) – you provide the lookup it calls
Import, duplicate detection, translation dashboard, assortment maintainer, snapshot contributor, thumbnail job	Cleansing of the source data (a named workstream)
Eight web screens and the till item-lookup data (Section 8)	Till screens (M6 renders your ItemCard data)

2. What you inherit

From	You use	How
17A kernel.api	ScopeContext, @CommandHandler, Handles, ProblemException, Ids, DomainEvent	As in the hello module
19A §4–5	AuditFacade, EventPublisher	One audit call and the events per command
19A §6 i18n	NfcConverter (automatic on I18nText columns), SearchSupport.orderByDisplayLanguage	Names normalised at write; search ordering
19A §8 sync	SnapshotContributor, SnapshotChangeLog.record(locations, table, rowId, op, applyFrom, urgent)	Section 7.3
19A §9 attachments	PresignService for image upload; object storage client	Images
19A §11–12	ConfigRegistry, @ScheduledJob	Thresholds; thumbnail and assortment jobs
M1 api/query	PartyQueries.getEntity, listLocations(entity), LookupRelationship; scope classes	Owner checks; assortment by location
M3 query (stub until doc 23)	PricingQueries.skusOnRetailList(entityId)	Assortment input
M5 query (stub until doc 25)	InventoryQueries.skusWithLots(locationId), hasAnyLot(skuId)	Assortment input; guards on unit and flag changes
17A §7 web shell	ItemCard, ScanField, DocumentHeader, ApprovalBar, ReasonCapture, StepUpDialog, generated client, useT	Section 8

3. Schema

Restated from doc 18 v0.2 part B (M2 tables) and doc 22 §3; adds supplier, location_assortment, duplicate_suspect and the has_printed_mrp flag (CR-22-1, CR-22-2, DR-7). Location db/migration/m2catalogue/.

```
-- V0001_catalogue.sql
CREATE TABLE catalogue.uom (uom_code varchar(10) PRIMARY KEY, name_en text NOT NULL, name_si text, name_ta
    text,
    is_weight boolean NOT NULL DEFAULT false);

CREATE TABLE catalogue.supplier (supplier_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL,
    name text NOT NULL, status text NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE','INACTIVE')),
    UNIQUE (owner_entity_id, name));

CREATE TABLE catalogue.sku (
    sku_id uuid PRIMARY KEY, sku_code varchar(12) NOT NULL UNIQUE, owner_entity_id uuid NOT NULL,
    status text NOT NULL DEFAULT 'DRAFT' CHECK (status IN ('DRAFT','LOCAL','SHARED','INACTIVE')),
    prior_status text CHECK (prior_status IN ('LOCAL','SHARED')), -- for reactivation
    short_name_en varchar(80) NOT NULL COLLATE en_icu, short_name_si varchar(80) COLLATE si_icu, short_name_ta
        varchar(80) COLLATE ta_icu,
    description_en text, description_si text, description_ta text,
    base_uom_code varchar(10) NOT NULL REFERENCES catalogue.uom,
    sold_by_weight boolean NOT NULL DEFAULT false, batch_tracked boolean NOT NULL DEFAULT false,
    expiry_tracked boolean NOT NULL DEFAULT false, has_printed_mrp boolean NOT NULL DEFAULT true,
    expiry_warning_days smallint, tax_category_id uuid NOT NULL,
    multi_mrp_policy text NOT NULL DEFAULT 'AUTO_LOWEST' CHECK (multi_mrp_policy IN
        ('AUTO_LOWEST','BARCODE_RESOLVED','PICKER')),
    origin_kind text NOT NULL DEFAULT 'PURCHASED' CHECK (origin_kind IN
        ('PURCHASED','REPACK_OUTPUT','LOCAL_SOURCED')),
    attributes jsonb NOT NULL DEFAULT '{}' CHECK (pg_column_size(attributes) <= 4096),
    CHECK (status <> 'SHARED' OR (short_name_si IS NOT NULL AND short_name_ta IS NOT NULL)), -- B-I1
    CHECK (NOT sold_by_weight OR base_uom_code = 'KG'),
    CHECK (NOT expiry_tracked OR batch_tracked)
);
CREATE INDEX sku_name_en_trgm ON catalogue.sku USING gin (short_name_en gin_trgm_ops);
CREATE INDEX sku_name_si_trgm ON catalogue.sku USING gin (short_name_si gin_trgm_ops);
CREATE INDEX sku_name_ta_trgm ON catalogue.sku USING gin (short_name_ta gin_trgm_ops);
CREATE INDEX sku_owner_status ON catalogue.sku (owner_entity_id, status);

CREATE TABLE catalogue.sku_uom_conversion (sku_id uuid NOT NULL REFERENCES catalogue.sku, uom_code varchar(10)
    NOT NULL REFERENCES catalogue.uom,
    factor_to_base numeric(14,6) NOT NULL CHECK (factor_to_base > 0), effective_from date NOT NULL, effective_to
    date,
    owner_entity_id uuid NOT NULL,
    EXCLUDE USING gist (sku_id WITH =, uom_code WITH =, daterange(effective_from,
        coalesce(effective_to,'infinity'::date), '[]') WITH &));

CREATE TABLE catalogue.sku_barcode (
    barcode varchar(48) NOT NULL, symbology text NOT NULL CHECK (symbology IN
        ('EAN13','EAN8','UPCA','GS1_128','GS1_DATAMATRIX','GS1_QR','INTERNAL')),
    sku_id uuid NOT NULL REFERENCES catalogue.sku, uom_code varchar(10) NOT NULL REFERENCES catalogue.uom,
    batch_id uuid, owner_entity_id uuid NOT NULL, status text NOT NULL DEFAULT 'ACTIVE' CHECK (status IN
        ('ACTIVE','RETIRED')),
    PRIMARY KEY (barcode, symbology, owner_entity_id);
-- B-I2: factory symbologies unique federation-wide; INTERNAL unique per owner
CREATE UNIQUE INDEX barcode_factory_unique ON catalogue.sku_barcode (barcode, symbology) WHERE symbology <>
    'INTERNAL' AND status = 'ACTIVE';

CREATE TABLE catalogue.sku_alias (alias_sku_id uuid PRIMARY KEY REFERENCES catalogue.sku, canonical_sku_id
    uuid NOT NULL REFERENCES catalogue.sku,
    effective_from date NOT NULL, owner_entity_id uuid NOT NULL);

CREATE TABLE catalogue.sku_image (image_id uuid PRIMARY KEY, sku_id uuid NOT NULL REFERENCES catalogue.sku,
    barcode varchar(48),
    owner_entity_id uuid NOT NULL, object_key_full text NOT NULL, object_key_thumb text, content_hash char(64)
    NOT NULL,
    status text NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('PENDING','ACTIVE','RETIRED')));
CREATE UNIQUE INDEX one_active_image ON catalogue.sku_image (sku_id, coalesce(barcode,''), owner_entity_id)
    WHERE status = 'ACTIVE';

CREATE TABLE catalogue.tag (tag_code varchar(40) PRIMARY KEY, name_en text NOT NULL, name_si text, name_ta
    text,
    governed boolean NOT NULL DEFAULT false, owner_entity_id uuid, CHECK (governed = (owner_entity_id IS
    NULL)));
CREATE TABLE catalogue.sku_tag (sku_id uuid NOT NULL REFERENCES catalogue.sku, tag_code varchar(40) NOT NULL
    REFERENCES catalogue.tag,
```

```

owner_entity_id uuid NOT NULL, PRIMARY KEY (sku_id, tag_code));

CREATE TABLE catalogue.tax_category (tax_category_id uuid PRIMARY KEY, code varchar(12) NOT NULL UNIQUE,
    name_en text NOT NULL, name_si text, name_ta text,
    owner_entity_id uuid NOT NULL);
CREATE TABLE catalogue.tax_rate (tax_category_id uuid NOT NULL REFERENCES catalogue.tax_category, rate_percent
    numeric(5,2) NOT NULL,
    effective_from date NOT NULL, effective_to date, owner_entity_id uuid NOT NULL,
    EXCLUDE USING gist (tax_category_id WITH =, daterange(effective_from,
        coalesce(effective_to,'infinity'::date), '[]') WITH &&));

CREATE TABLE catalogue.batch (
    batch_id uuid NOT NULL, sku_id uuid NOT NULL REFERENCES catalogue.sku, batch_no varchar(40) NOT NULL,
    supplier_id uuid, manufacture_date date, expiry_date date, printed_mrp numeric(14,2),
    origin_document_id uuid, is_synthetic boolean NOT NULL DEFAULT false, corrects_batch_id uuid,
    status text NOT NULL DEFAULT 'REGISTERED' CHECK (status IN ('REGISTERED','SUPERSEDED')),
    created_at timestampz NOT NULL DEFAULT now(), owner_entity_id uuid NOT NULL,      -- registering entity
    (for RLS)
    PRIMARY KEY (created_at, batch_id),
    UNIQUE (sku_id, coalesce(supplier_id,'00000000-0000-0000-0000-000000000000'::uuid), batch_no, created_at)
) PARTITION BY RANGE (created_at);
CREATE INDEX batch_lookup ON catalogue.batch (sku_id, batch_no);
-- batch is global identity: readable by every authenticated scope (policy shared_read USING (true) for
    SELECT);
-- writes by the registering module only.

CREATE TABLE catalogue.location_assortment (location_id uuid NOT NULL, sku_id uuid NOT NULL, reasons text[]
    NOT NULL,
    sell_through boolean NOT NULL DEFAULT false, since timestampz NOT NULL DEFAULT now(), owner_entity_id uuid
    NOT NULL,
    PRIMARY KEY (location_id, sku_id));

CREATE TABLE catalogue.duplicate_suspect (suspect_id uuid PRIMARY KEY, sku_a uuid NOT NULL, sku_b uuid NOT
    NULL, score numeric(4,3) NOT NULL,
    reasons text[] NOT NULL, status text NOT NULL DEFAULT 'OPEN' CHECK (status IN
        ('OPEN','MERGED','DISMISSED')),
    decided_by uuid, decided_at timestampz, owner_entity_id uuid NOT NULL, CHECK (sku_a < sku_b), UNIQUE
    (sku_a, sku_b));

-- RLS: 17A 6.3 template on every table, plus:
CREATE POLICY shared_read ON catalogue.sku FOR SELECT TO app_rw USING (status = 'SHARED');
-- uom, tag (governed), tax_category, tax_rate, sku_alias: readable by all authenticated scopes.
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA catalogue TO app_rw; -- batch: INSERT + UPDATE(status)
    only

```

3.1 Seed data

```

# seed/m2/uom.yaml
- {code: EA, en: each, si: ..., ta: ..., weight: false} - {code: KG, en: kilogram, weight: true} - {code:
    G, weight: true}
- {code: L} - {code: ML} - {code: CASE} - {code: PKT} - {code: BAG} - {code: BOX} - {code: DOZ}
# seed/m2/tax.yaml
- {code: STD, en: Standard VAT, rates: [{percent: 18.00, from: 2024-01-01}]}
- {code: ZERO, en: Zero-rated, rates: [{percent: 0.00, from: 2000-01-01}]}
- {code: EXEMPT, en: Exempt, rates: [{percent: 0.00, from: 2000-01-01}]}
# seed/m2/tags.yaml (governed)
- core-range - rice - dairy - frozen - fresh-produce - dry-goods - household - controlled-price
# seed/m2/permissions.yaml (appended to the M1-owned catalogue file)
- cat.sku.create (F) - cat.sku.create_local (ENTITY) - cat.sku.promote (F) - cat.sku.deactivate (ENTITY)
- cat.barcode.manage (ENTITY) - cat.image.manage (ENTITY) - cat.tag.manage (ENTITY; governed tags F)
- cat.tax.publish (F, mfa) - cat.batch.correct (ENTITY, mfa) - cat.supplier.manage (ENTITY) -
    cat.import.run (ENTITY)
# seed/m2/transliteration-v1.csv (Sinhala and Tamil grapheme -> Latin key; curated with the language
    reviewers)

```

4. Package layout

```
lk.coopfed.knoweb.m2catalogue/                                     @ApplicationModule(allowedDependencies = {"kernel",
    "m1party::api", "m1party::query",                                     "m3pricing::query",
                                                                    "m5inventory::query"})
api/                      commands (17), events (9 families), RegisterBatch (internal command, exported for M4/M5), value
    types (SkuCode, Gtin)
query/                    CatalogueQueries (lookup, search, get, list, alias), BatchQueries, TagQueries, TaxQueries,
    AssortmentQueries
internal/
    sku/                  Sku, SkuRepository, Create/Activate/Update/Promote/Merge/Deactivate handlers,
        SkuCodeGenerator
    unit/                  Uom, Conversion, DefineConversionHandler
    barcode/              Barcode, BarcodeRepository, handlers, GtinParser (validates check digit; splits AI 01/10/17
        already parsed by the till)
    image/                 Image, AttachImageHandler, ThumbnailJob
    tag/                   Tag, handlers
    tax/                   TaxCategory, TaxRate, PublishTaxRateHandler
    batch/                 Batch, BatchRepository, RegisterBatchHandler, CorrectBatchHandler
    supplier/              Supplier, RegisterSupplierHandler
    steward/               ImportService, RowValidator, DuplicateScorer, Transliterator, DuplicateSuspectRepository,
        TranslationDashboard
    assortment/            AssortmentMaintainer (event consumers), AssortmentRepository
    snapshot/              CatalogueSnapshotContributor
    queries/               *QueriesImpl
web/                      SkuController, BarcodeController, ImageController, TagController, TaxController,
    BatchController,
        SupplierController, ImportController, StewardController (implement generated m2catalogue.yaml)
```

5. API slice: openapi/m2catalogue.yaml

```
paths:
  /v1/catalogue/skus:
    post: createSku (cat.sku.create | create_local)  get: listSkus
    / searchSkus (q, lang, status[], tag[], cursor)
  /v1/catalogue/skus/{skuId}:
    get: getSku  patch: updateSku
  /v1/catalogue/skus/{skuId}/activate:
    post: activateLocalSku | activateSharedSku (body.target =
        LOCAL|SHARED)
  /v1/catalogue/skus/{skuId}/promote:
    post: promoteToShared (cat.sku.promote)  body: {mode: NEW |
        MERGE, targetSkuId?}
  /v1/catalogue/skus/{skuId}/deactivate:
    post: deactivateSku  /reactivate: post
  /v1/catalogue/skus/{skuId}/conversions:
    post: defineConversion
  /v1/catalogue/skus/{skuId}/barcodes:
    post: registerBarcode  delete /{barcode}: retireBarcode  post
        /{barcode}/link: linkBarcodeToBatch
  /v1/catalogue/skus/{skuId}/images:
    post: attachImage (returns presigned PUT + imageId)  delete
        /{imageId}: retireImage
  /v1/catalogue/skus/{skuId}/tags:
    put: tagSku (replace set)
  /v1/catalogue/lookup:
    get: lookupByBarcode (barcode | gtin, lot?, expiry?, symbology,
        locationId)
  /v1/catalogue/tags:
    post: defineTag  get: listTags
  /v1/catalogue/tax/categories:
    post: defineTaxCategory  get
  /v1/catalogue/tax/rates:
    post: publishTaxRate (cat.tax.publish, mfa)  get: rateInForce
        (category, date)
  /v1/catalogue/batches/{batchId}:
    get: getBatch  post /correct: correctBatch (cat.batch.correct,
        mfa)
  /v1/catalogue/batches:
    get: listBatches (skuId, batchNo?, expiringBefore?)
  /v1/catalogue/suppliers:
    post: registerSupplier  get
  /v1/catalogue/import:
    post: importSkus (multipart; mode=validate|commit) ->
        ImportReport
  /v1/catalogue/steward/suspects:
    get: listSuspects  post /{id}/resolve: resolveDuplicateSuspect
  /v1/catalogue/steward/dashboard:
    get: stewardDashboard
components.schemas:
  LookupResult: { skuId, skuCode, nameEn, nameSi?, nameTa?, fallback: {si: bool, ta: bool}, uomCode,
    factorToBase,
        batch?: {batchId, batchNo, expiryDate?, printedMrp?}, thumbKey?, sellThrough: bool,
        hasPrintedMrp, soldByWeight }
  ImportReport: { rows: int, accepted: int, errors: [{row, field, type, params}], suspects: [{row,
    matchedSkuId, score, reasons}] }
# example - lookup by 2D data resolving a batch without a registered batch barcode
GET
  /v1/catalogue/lookup?gtin=4791234567890&lot=B2411A&expiry=2027-03-31&symbology=GS1_DATAMATRIX&locationId=...
{ "skuId": "...", "skuCode": "SKU-7Q2K9", "nameEn": "Milk powder 400g", "uomCode": "EA", "factorToBase": 1,
  "batch": { "batchId": "...", "batchNo": "B2411A", "expiryDate": "2027-03-31", "printedMrp": 1080.00 },
  "thumbKey": "images/.../t.webp" }
```

6. Handler specifications

Command	Guards (in order)	Mutation	Audit	Events
CreateSku	owner = caller scope (F for shared path); English name; base uom exists; tax category exists; weight rule; expiry rule; attributes size; duplicate scoring (suspects recorded, never blocking)	insert DRAFT; code from SkuCodeGenerator	SKU_CREATE_D	sku.created.v1; duplicate.suspected.v1 per suspect
ActivateLocalSku	owner; status DRAFT; barcode or code-lookup allowed by config	LOCAL	SKU_ACTIVATED	sku.activated.v1
ActivateSharedSku	caller F; DRAFT; three names; governed tags only; CASE conversion if a case barcode exists	SHARED; owner = Federation	SKU_SHARED	sku.shared.v1
UpdateSku	owner (F for SHARED); changing base_uom or tracking flags requires !inventory.hasAnyLot(sku); SHARED keeps three names	fields; audited diff	SKU_UPDATE_D	sku.updated.v1
PromoteToShared (NEW)	caller F; LOCAL; names complete; governed tags	owner → F; status SHARED; alias(origin sku → itself, effective today) so the origin entity's history resolves	SKU_PROMOTED	sku.promoted.v1
MergeIntoShared	caller F; source LOCAL; target SHARED; same base uom; no unit conflict	alias(source → target); move ACTIVE barcodes (retire colliding); source INACTIVE with prior_status LOCAL	SKU_MERGE_D	sku.merged.v1 (M5 re-points lots)
DeactivateSku	owner; status LOCAL or SHARED; reason	INACTIVE; prior_status kept; assortment rows get sell_through = true where lots exist	SKU_DEACTIVATED	sku.deactivated.v1
ReactivateSku	owner; INACTIVE; reason	prior_status	SKU_REACTIVATED	sku.reactivated.v1
DefineConversion	owner; uom exists; factor > 0; sku not sold_by_weight for CASE-type units; exclusion constraint	insert effective row	CONVERSION_DEFINED	conversion.defined.v1
RegisterBarcode	sku active; symbology valid; check digit valid for EAN/UPC; uniqueness per rule; INTERNAL only for own SKUs	insert	BARCODE_REGISTERED	barcode.registered.v1
RetireBarcode / LinkBarcodeToBatch	owner; batch belongs to sku	status / batch_id	BARCODE_RETIRED / LINKED	barcode.retired/linked.v1
AttachImage	owner (or local override on SHARED); one active per key	insert PENDING; presign returned; on upload event: verify hash, ThumbnailJob, ACTIVE	IMAGE_ATTACHED	image.attached.v1 (after thumbnail)
DefineTag / TagSku	governed tags F only; local tags owner; TagSku: governed tags on SHARED only by F	rows	TAG_DEFINED / SKU_UPDATED	tag.defined.v1; sku.updated.v1
PublishTaxRate	caller F; MFA; apply_from ≥ today; exclusion	close current at apply_from – 1; insert	TAX_RATE_PUBLISHED	tax_rate.published.v1 (apply_from)
RegisterBatch (internal)	sku not INACTIVE; (sku, supplier, batch_no) lookup; MRP present when has_printed_mrp; expiry when expiry_tracked; synthetic naming when !batch_tracked	return existing or insert; duplicate-suspect check on near-identical batch numbers	BATCH_REGISTERED	batch.registered.v1
CorrectBatchMrp / Expiry	caller owns a lot of the batch (M5 query) or F; MFA; reason; new value differs	insert replacement batch with corrects_batch_id; old SUPERSEDED	BATCH_CORRECTED	batch.corrected.v1
RegisterSupplier	owner	insert	SUPPLIER_REGISTERED	supplier.registered.v1
BulkImportSkus	per row: CreateSku guards; sheet row hash for idempotency	validate: report only; commit: one transaction per row	per row	per row

Command	Guards (in order)	Mutation	Audit	Events
ResolveDuplicateSuspect	F for pairs involving SHARED; owner for local-local	MERGED via MergeIntoShared, or DISMISSED	SUSPECT_RE SOLVED	sku.merged.v1 or duplicate.dismissed.v1

6.1 Duplicate scoring

```
// steward/DuplicateScorer.score(candidate, existing) -> Score(value 0..1, reasons[])
key(s)      = nfc(s) -> lowercase -> strip punctuation -> transliterate(si|ta -> latin, table v1) -> tokens
brand(t)    = first token if in brand-list (governed tag brand:* or a seeded brand file), else null
pack(t)     = tokens matching (\d+(\.\d+)?)(kg|g|l|ml|pkt|pcs) normalised to base unit
score       = 0.55 * jaroWinkler(key(candidate.nameEn), key(existing.nameEn))
              + 0.20 * [brand equal] + 0.15 * [pack equal within 2%] + 0.10 * [same base uom]
              + 0.30 if any barcode equal (capped at 1.0) // a shared barcode is near-certain
suspect     = score >= config catalogue.duplicate_threshold (0.85) and not previously DISMISSED for this pair
// applied on CreateSku, BulkImport (row vs catalogue and row vs row), and RegisterBatch (batch_no edit
              distance <= 1
// for the same sku and supplier -> batch-level suspect flagged to the steward, batches still both registered)
```

6.2 Import

```
// steward/ImportService.run(sheet, mode, ctx)
rows = parse(sheet) // xlsx or csv; columns: code?, name_en, name_si,
                    name_ta, base_uom, tax, flags, tags, barcodes[], case_factor, image_ref
for r in rows:
    key = hash(r.rawCells); if importLog.seen(key) -> outcome DUPLICATE_ROW (idempotent re-upload)
    errors = RowValidator.validate(r) // each error = {row, field, messageId, params}
    suspects = scorer.against(catalogue).plus(scorer.withinSheet(rows))
    if mode == VALIDATE: report.add(r, errors, suspects); continue
    if errors.isEmpty(): tx { CreateSku(...); register barcodes; define CASE conversion; importLog.record(key) }
    ; report.accepted++
    else report.add(r, errors)
return report // rendered by the import screen; downloadable as a
              sheet
```

7. Queries, assortment and snapshot

Query	Implementation	Cache
LookupByBarcode	1) exact (barcode, symbology) ACTIVE row → sku, uom, batch link; 2) else if gtin+lot: sku by gtin barcode then batch_lookup by (sku, lot); 3) else INTERNAL code in owner scope; returns LookupResult with sell_through from assortment	Per-instance by (barcode, symbology) 60 s; invalidated by barcode.*
SearchSku	trigram similarity on the three names OR prefix on sku_code; filter by scope (SHARED or OWN); order by display-language collation then similarity; paged	–
GetSku / ListSkus	joins conversions, barcodes, tags, images (thumb keys), alias	SHARED SKUs by id: per-instance 10 min; invalidated by sku.*
ResolveAlias	follow alias chain (max depth 3)	10 min
ListBatches / GetBatch	by sku, batchNo, expiring window; status	–
TaxQueries.rateInForce	exclusion-indexed range lookup	Per-instance 1 h; invalidated by tax_rate.published
AssortmentQueries.get(location)	location_assortment rows	–
StewardDashboard	counts: SKUs missing si/ta by tag and owner; OPEN suspects; ACTIVE barcodes on INACTIVE SKUs; SHARED without image by tag	Materialised nightly + on demand

7.2 Assortment maintainer

```
// assortment/AssortmentMaintainer - event consumers, each recomputes affected (location, sku) rows
@EventConsumer(types={"price_list.published.v1"}) -> for each location of the MPCs: add SKUs on the
list (reason RETAIL_LIST), remove those no longer listed unless another reason holds
@EventConsumer(types={"stock.moved.v1", "lot.negative.v1"}) -> location: add sku (reason LOT) when on-hand >
0; keep with sell_through if INACTIVE
@EventConsumer(types={"sku.activated.v1", "sku.deactivated.v1", "sku.reactivated.v1", "sku.merged.v1"})
-> owner entity's locations: LOCAL reason;
INACTIVE -> sell_through where LOT reason
exists else remove
@EventConsumer(types={"sku.updated.v1"}) when tags change -> core-range tag: all locations (worker
fan-out, batched)
// every change to location_assortment -> SnapshotChangeLog.record([location], "sku", skuId, UPSERT|TOMBSTONE,
null, false)
// plus for the SKU's barcodes, conversions, images, tags: UPSERT rows so the till receives the whole item
// nightly AssortmentReconcileJob recomputes from the three source queries and repairs drift (REVIEW event on
any diff)
```

7.3 Snapshot contributor

```
// snapshot/CatalogueSnapshotContributor implements SnapshotContributor
tables():
["sku", "sku_uom_conversion", "sku_barcode", "sku_image", "tag", "sku_tag", "tax_category", "tax_rate", "batch"]
rowsFor(locationId, table, rowIds):
sku -> for ids in assortment: {skuId, code, names(3), flags, baseUom, taxCategoryId, multiMrpPolicy,
sellThrough, attributes? no}
batch -> ids supplied by M5's lot contributor (batches with a lot at the location): {batchId, skuId,
batchNo, expiry, printedMrp, status}
tax_rate -> all rows with effective_to null or future; applyFrom = effective_from
others -> rows for the assortment's SKUs; images = thumb keys only
fullSnapshotScope(locationId): assortment sku ids (used by the nightly full-snapshot builder)
```

8. Screens

```
Price book (catalogue browser)
+-----+
| ScopeBanner: Kandy South MPCs (M042) | si | mpcs-admin |
+-----+
| [search in any script _____] status [Shared+Local v] tag [any v] [+ New item] [Upload] |
| [thumb] SKU-7Q2K9 Milk powder 400g / (si) / (ta EN*) EA STD shared barcodes 2 > |
| [thumb] SKU-3F8A1 Red rice 1kg (local) KG ZERO local barcodes 1 > |
| * EN tag marks a missing translation (fallback shown) |
+-----+
Product card: header (code, status chip, owner) | names block (3 fields, completeness marks) | units &
conversions table |
barcodes table (symbology, unit, batch link, status) | images (full + thumb, override badge) | tags
(governed vs local) |
attributes (key/value, bounded) | batches tab (list with expiry and MRP; Correct... opens ReasonCapture +
```

StepUp) |
ApprovalBar by status: [Activate] [Promote...] [Deactivate...]

Stock list upload: dropzone -> Validate -> report table (row, field, reason in shop language) + suspects panel
(candidate card | matched card | [Merge] [Distinct]) -> [Commit accepted rows]
Duplicate suspects: list sorted by score; side-by-side cards; decisions remembered
Translation completeness: matrix tag x language with counts; [Export missing] [Import translations]

Screen	Components	Message ids (m2.)	Notes
Price book	ScopeBanner, list with ItemCard rows, filters	sku.*	Search box accepts any script; results ordered by display language
Product card	DocumentHeader-style card, tabs, ApprovalBar, ReasonCapture, StepUpDialog, AttachmentCapture (images)	sku.*, barcode.*, image.*	Image upload uses the presigned PUT directly from the browser
Stock list upload	dropzone, report table, suspects panel	import.*	Report downloadable
Duplicate suspects	two-pane cards	steward.*	–
Translation completeness	matrix, export/import	steward.*	–
Batch card	card, ReasonCapture, StepUpDialog	batch.*	Shows corrects/superseded chain
Tax rates	list, form with apply_from	tax.*	F only
Item lookup (till)	M6 renders ItemCard from LookupResult	–	Data contract is LookupResult; M6 owns the screen

9. Tests

Layer	Cases (fixtures in src/test/resources/m2/)	Target
Unit	Every guard of Section 6; SkuCodeGenerator uniqueness under concurrency; GtinParser check digits; conversion effective ranges; synthetic batch naming; scorer symmetry and NFC invariance	All guards
Contract	Every operation; lookup by gtin+lot without a batch barcode; ImportReport shape; presign round trip for images	Every operation
State machine	SKU, barcode, batch, tax rate transitions incl. promote NEW vs MERGE and the alias consequences	All transitions
Property-based (jqwik)	Import sheets: each row yields one DRAFT or one report line; re-import idempotent; MergeIntoShared never leaves an ACTIVE barcode colliding; assortment membership equals the union of the three sources after any event sequence	1000 cases each
Data quality	The name test set (doc 19 §5.4) with labelled duplicate/distinct pairs: precision ≥ 0.9 , recall ≥ 0.8 at the default threshold	Measured
RLS matrix	SHARED visible to all; LOCAL OWN plus Federation view; batches readable by all, writable by registering module; assortment OWN	Full matrix
Scenario (integration)	Flows 6.1–6.8 of doc 22; GRN line without MRP for has_printed_mrp SKU blocked (through M4's stub); MRP correction re-points lots (through M5's stub event)	Eight flows
Snapshot	Contributor output for a fixture shop with 50 SKUs; a price-list publication produces exactly the expected change-log rows; nightly reconcile finds injected drift	Three cases
Performance	Import 20,000 rows with validation under 5 min; search under 100 ms on 500,000 SKUs; snapshot build for 6,000 SKUs under 5 s	Three targets
Web	Playwright: upload sheet → validate → resolve a suspect → commit → activate shared; product card image attach	Two flows

10. Tickets in build order

#	Ticket	Est. (days)	Done when
M2-01	Scaffold; migration V0001 with RLS, partitioned batch; seeds (uom, tax, tags, permissions); schema-ownership check	1.5	Migrations and seeds apply
M2-02	SKU aggregate: create, activate (local/shared), update, deactivate/reactivate; code generator; search and get queries	2.5	Search in three scripts works on fixtures
M2-03	Units and conversions; tags; tax categories and rates with apply_from	1.5	Exclusion constraints tested
M2-04	Barcodes: registry, uniqueness rules, GtinParser, lookup incl. gtin+lot batch resolution	2	Lookup contract tests
M2-05	Batch: RegisterBatch internal command (existing-or-create, synthetic), corrections with replacement batch; supplier table	2	Correction chain; M5 stub consumes batch.corrected
M2-06	Images: attach with presign, verifier hook, ThumbnailJob, local override	1.5	Thumb produced and keyed
M2-07	Promotion and merge with alias; barcode migration; M5 stub re-point	1.5	Flow 6.3
M2-08	Steward: import service, row validator, transliterator, duplicate scorer, suspects, translation dashboard	3.5	Flow 6.1, 6.2; data-quality targets
M2-09	Assortment maintainer consumers and reconcile job; snapshot contributor; change-log producers	2.5	Snapshot tests; reconcile finds drift
M2-10	Web screens (eight) and Playwright flows	3	Both flows green
M2-11	Contract freeze: m2catalogue.yaml v1; api/query review with M3 and M5 developers; doc 33 slice	0.5	M3 acknowledges

Total about 22 developer-days (about 4.5 weeks for one developer). Freeze the contract after M2-05 (week 2) so M3 and M5 can design against api while the steward tooling and screens finish.

11. Open points: what to assume, whom to ask

Point	Assume	Decider	If different
Minimal supplier table, assortment table, has_printed_mrp (doc 22 DR-1, DR-2, DR-7)	Adopted; CRs to doc 18 raised	Architecture review	–
Deactivation sell-through (DR-3)	On: INACTIVE keeps selling lots on hand	Federation admins	Assortment removes INACTIVE rows immediately
Duplicate threshold and table (DR-4)	0.85; transliteration v1 in the repo	Data steward (unnamed – T-03)	Config change; new table version
Local image override (DR-5)	Allowed	Federation	Remove the override path
Label content (DR-6)	Code + weight/batch	Federation, M3, M6	Never price; if overruled, M3 must add a label-price ceiling check
M3 skusOnRetailList and M5 skusWithLots / hasAnyLot	Stubs returning fixture data until docs 23/25 land	M3, M5 developers	Swap the stubs; tests already assert the calls
Brand list for scoring	Seeded from the import sheet's brand column; governed tags brand:* later	Data steward	–

End of document. 22A v0.1; the repository's `m2catalogue/README.md` and its tests become the living version once M2 is merged.